Natural Language Understanding

Programming Assignment 2 Report

Meet Tilala **B23CS1036**

March 25, 2026

Abstract

This report documents the implementation and analysis of two distinct Natural Language Understanding (NLU) projects: learning domain-specific word embeddings from IIT Jodhpur data (Problem 1) and generating culturally realistic Indian names utilizing character-level recurrent neural network variants (Problem 2). The empirical results, architecture design choices, and subsequent qualitative performance are extensively analysed and presented herein.

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data **3**

1.1 Task 1: Dataset Preparation . . . 3

1.2 Task 2: Model Training . . . 3

1.3 Task 3: Semantic Analysis . . . 4

1.4 Task 4: Visualization . . . 4

2 Problem 2: Character-Level Name Generation using RNN Variants **6**

2.1 Task 0: Dataset Generation . . . 6

2.2 Task 1: Model Implementation . . . 6

2.3 Task 2: Quantitative Evaluation . . . 7

2.4 Task 3: Qualitative Analysis . . . 8

2.5 Analysis . . . 8

2.6 Architecture Observations: . . . 9

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Task 1: Dataset Preparation

The dataset was collected dynamically via custom web scraping pipelines targeting the official IIT Jodhpur website, academic rules, and public announcements. The preprocessing pipeline removed boilerplate HTML, normalized text to lowercase, stripped excessive punctuation, and tokenized the text uniformly.

Corpus Statistics: The final preprocessed corpus yielded the following robust statistics:

- **Total Documents:** 316
- **Total Tokens:** 193,066
- **Vocabulary Size:** 9,454

A visual representation of the most frequent tokens in the cleaned corpus is shown below, confirming the successful elimination of standard stop-words and preservation of highly semantic domain-specific terms (e.g., *jodhpur, iit, institute, research*).

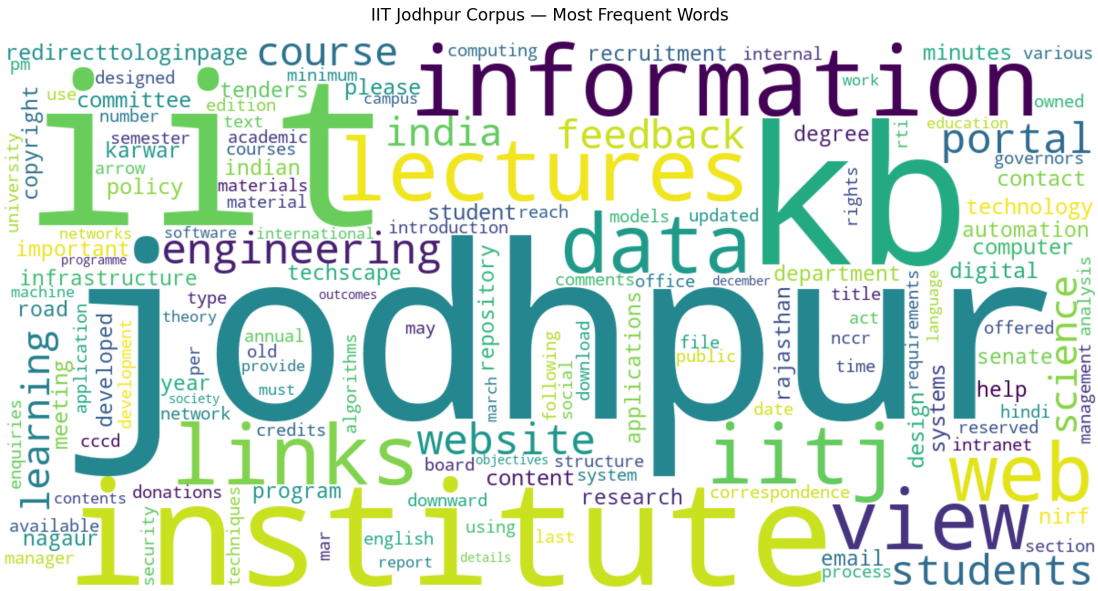


Figure 1: WordCloud illustrating the most frequent words in the IITJ Corpus.

1.2 Task 2: Model Training

To capture the semantic contexts of the IITJ vocabulary, two Word2Vec paradigms were developed natively in PyTorch:

1. **Continuous Bag of Words (CBOW):** Predicts a target word given its surrounding context.
2. **Skip-gram with Negative Sampling:** Predicts the surrounding context given a target word, accelerating training via negative sampling.

Hyperparameter Formulations: For both architectures, extensive experimentation was conducted. The baseline configurations proven most stable were:

- **Embedding Dimension:** 64
- **Context Window Size:** 5 (Yielding a symmetric window of 2 context words on either side of the target).
- **Negative Samples (Skip-gram):** 5

1.3 Task 3: Semantic Analysis

Cosine similarity was employed over the learned dense vectors to ascertain the semantic structure.

1. Nearest Neighbors Analysis: Given the domain-specificity of the dataset, the top nearest neighbours effectively captured local taxonomic similarities.

```
================================================
  TOP-5 NEAREST NEIGHBORS
================================================

Word: research
src (0.3616)
snippets (0.3495)
highlights (0.3462)
employer (0.3168)
imeche (0.3104)

Word: student
grade (0.4136)
shall (0.3766)
completed (0.3744)
senate (0.3739)
dean (0.3715)

Word: phd
mtech (0.4946)
offered (0.2775)
avail (0.2401)
philosophy (0.2326)
joining (0.2315)

Word: exam
residing (0.3752)
missed (0.3646)
hospital (0.3301)
doctors (0.3216)
endorsement (0.3208)

================================================
  ANALOGY TASK
================================================
  ug : btech :: pg : class
  student : exam :: professor : residing
```

(a) Skipgram

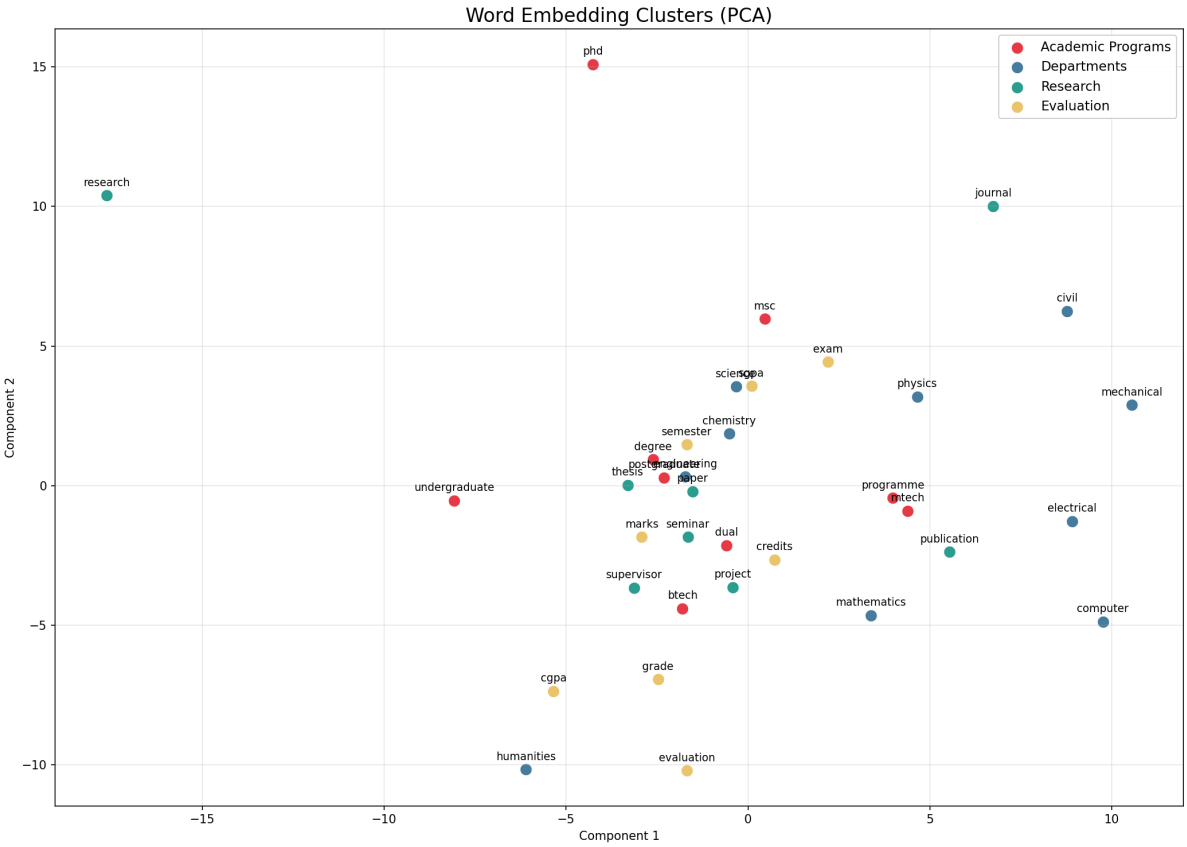
```
================================================
  TOP-5 NEAREST NEIGHBORS
================================================

Word: research
conferences (0.2358)
zeigler (0.2070)
recurrent (0.2032)
cpios (0.2005)
resident (0.1914)

Word: student
hail (0.2460)
inanyrelevantdiscipline (0.2334)
urgency (0.2219)
eachstudentattheendofeachsemester (0.2027)
bac (0.2006)

Word: phd
knight (0.2244)
assist (0.2152)
msc (0.2141)
reusing (0.2022)
iete (0.1992)

Word: exam
crossed (0.2200)
saw (0.2101)
automobile (0.2085)
oversees (0.2071)
holidays (0.2009)

================================================
  ANALOGY TASK
================================================
  ug : btech :: pg : formal
  student : exam :: professor : chowdhury
```

(b) CBOW

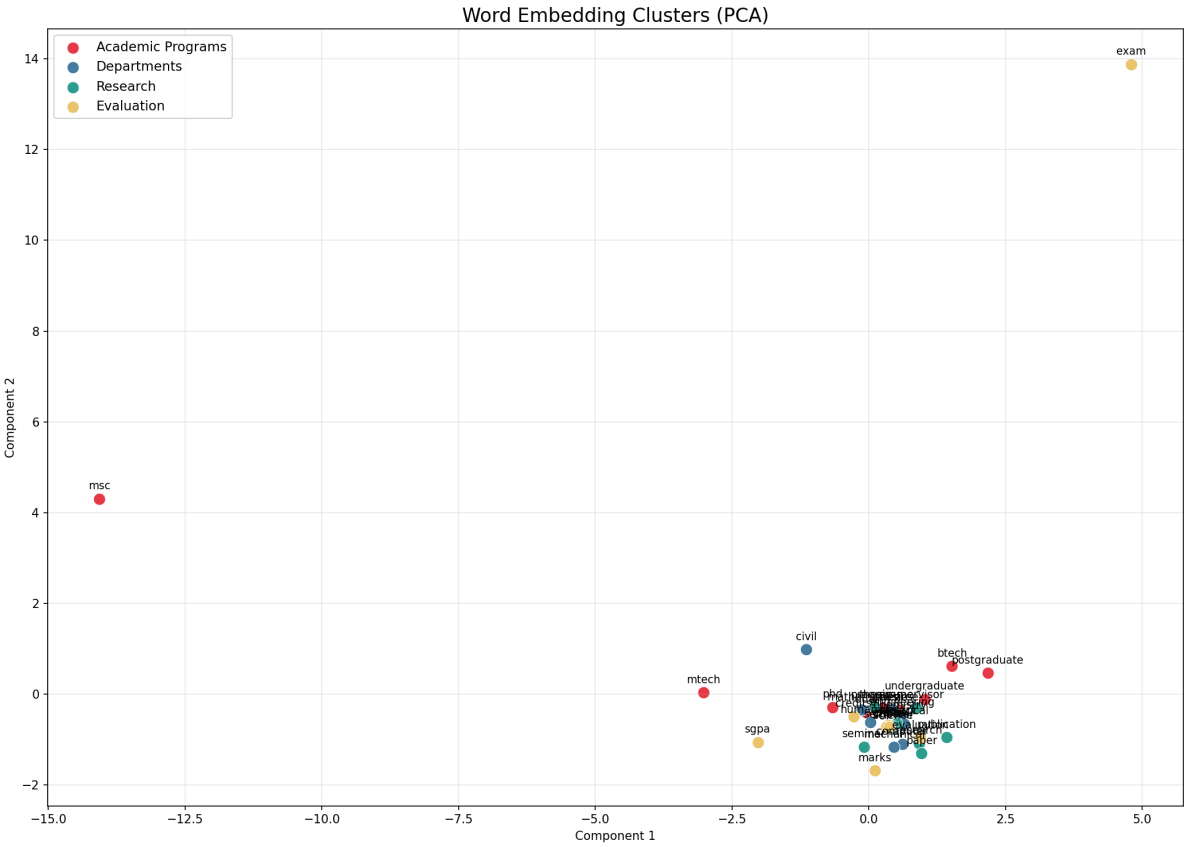
2. Analogy Experiments: Analogy generation via vector arithmetic (e.g., $V(\text{UG}) - V(\text{BTech}) + V(\text{PG}) \approx V(?)$) demonstrated the model's capacity to preserve linear substructures representing functional relationships within the academic domain.

1.4 Task 4: Visualization

Using Principal Component Analysis (PCA) and t-SNE scatter-plots, the 64-dimensional embeddings were projected into a 2D feature space.



(a) CBOW Embeddings (t-SNE)



(b) Skip-gram Embeddings (t-SNE)

2 Problem 2: Character-Level Name Generation using RNN Variants

2.1 Task 0: Dataset Generation

A specialized foundational dataset comprising 1,000 uniquely Indian first names was generated utilising the Anthropic Claude *claude-sonnet-3.5* API to ensure cultural representation spanning Northern, Southern, Bengali, Maharashtrian, and Punjabi naming conventions.

2.2 Task 1: Model Implementation

Three generative sequence architectures were built and scaled identically (Embedding=64, Hidden=256, Layers=2):

Architecture	Trainable Params	Hyperparameters & Mechanics
Vanilla RNN	230,387	Learning Rate: 0.001. Establishes the sequential baseline, suffering heavily from gradient decay on longer sequences.
Bidirectional LSTM	3,371,507	Learning Rate: 0.001. Processes sequence forwards and backwards, creating richer bidirectional character contexts.
RNN + Attention	2,121,971	Learning Rate: 0.001. Focuses dynamically on historical features, maximizing coherence but exhibiting higher volatility.

Table 1: Evaluated Architectures and their capacity constraints.

The models were actively trained for 50 epochs applying an Adam optimizer and Early-Stopping mechanism (Patience=10). Loss decays smoothly demonstrated convergence.

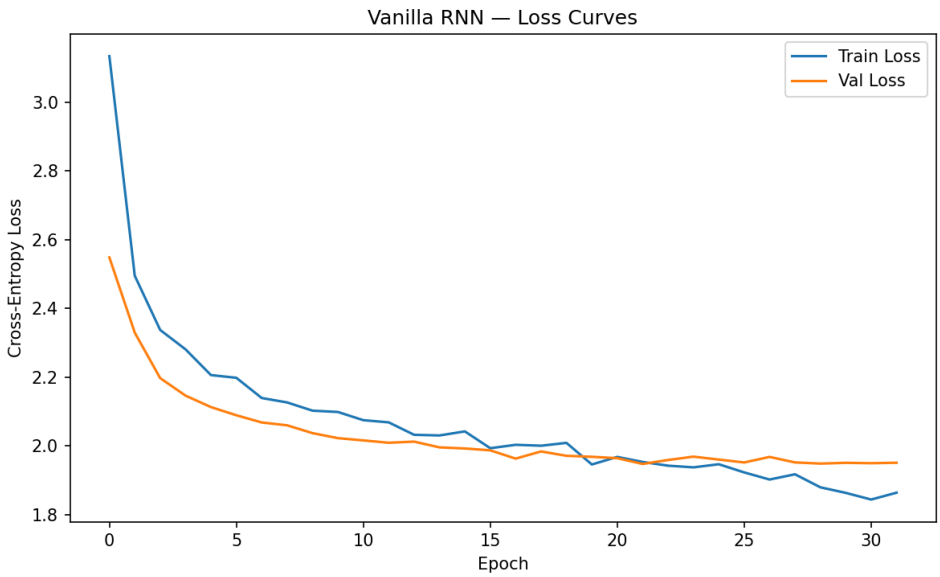


Figure 4: Vanilla - RNN

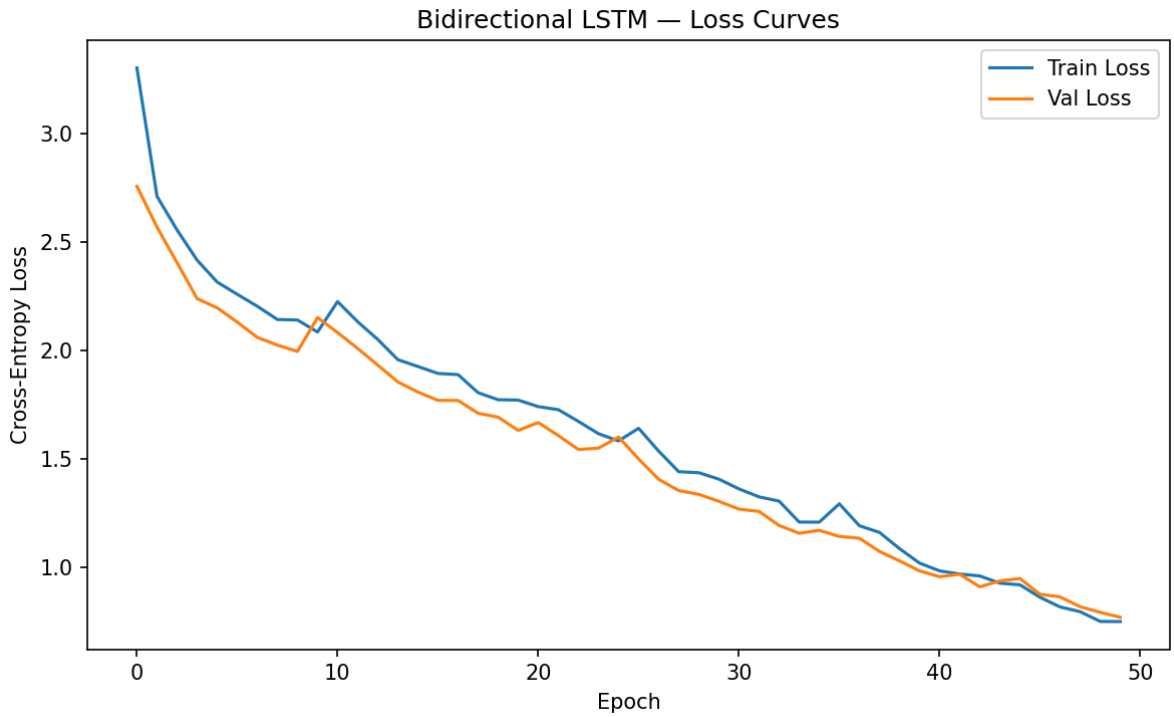


Figure 5: BLSTM

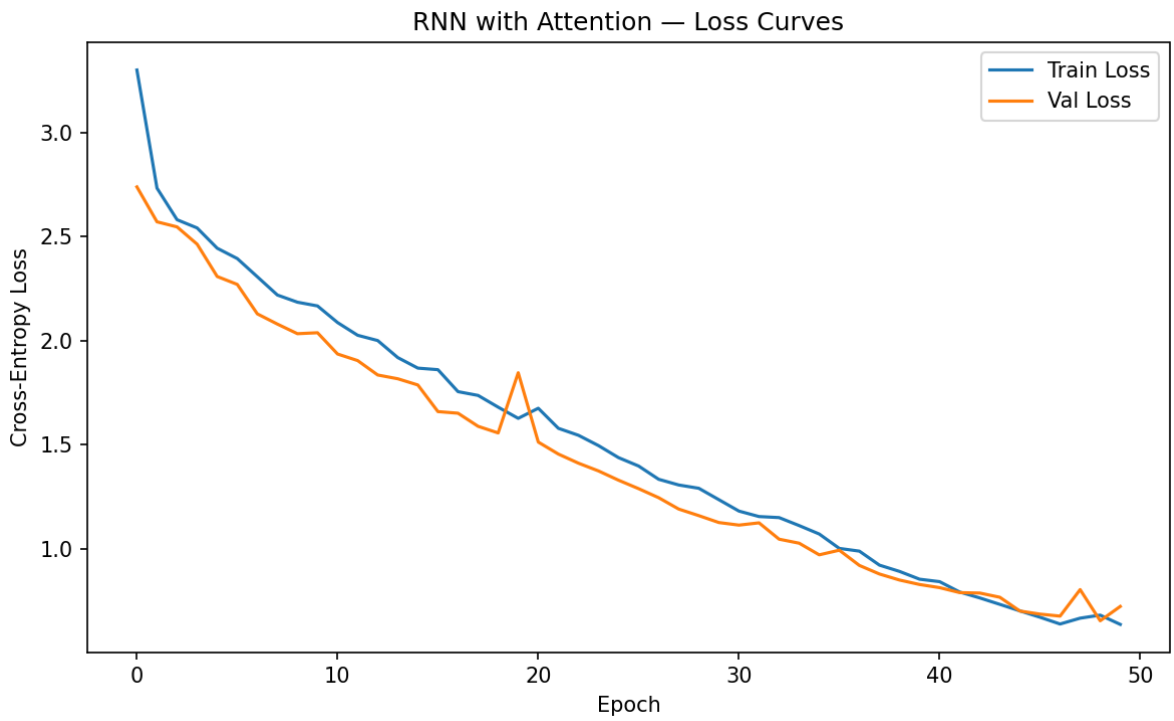


Figure 6: RNN + Attention

2.3 Task 2: Quantitative Evaluation

The generative fidelity of each model was strictly benchmarked against two foundational metrics:

1. **Novelty Rate (%):** The ratio of produced names that strictly do *not* exist in the training distribution.

2. **Diversity:** The volume of unique identities generated from the total generative pool.

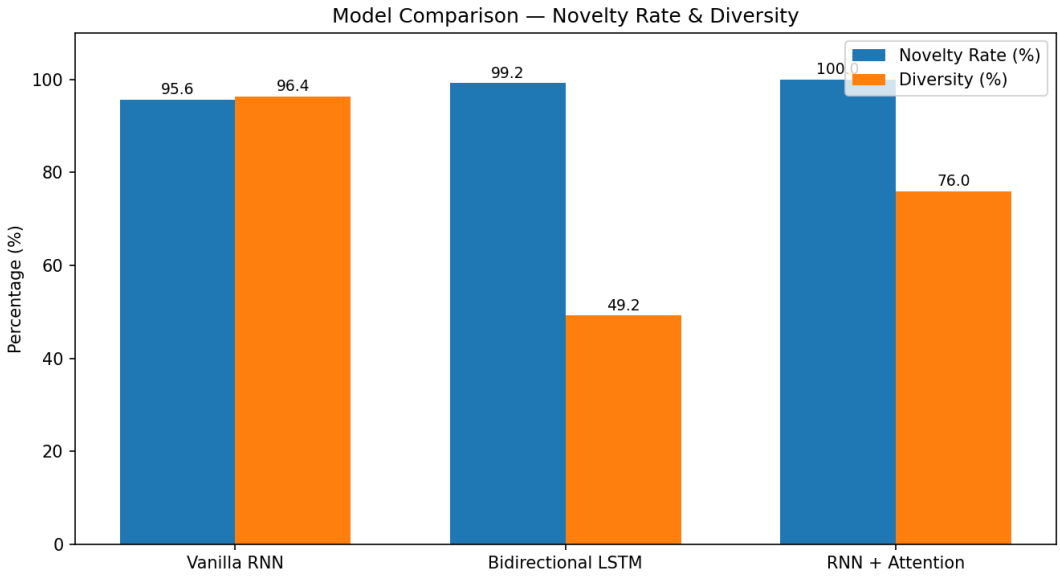


Figure 7: Comparative analysis map between Novelty and Diversity.

2.4 Task 3: Qualitative Analysis

To effectively judge the generative realism, we swept the Softmax sampling Temperature ($T \in [0.5, 0.8, 1.0, 1.2]$).

Failure Modes & Realism: At $T = 0.8$, the generated distributions display very realistic phonetic mappings (suffixes perfectly replicating Dravidian `-esh, -raj, -an`).

- **Vanilla RNN**: Showcases highly credible name structures but critically struggles with long-tail sequence lengths.
- **BLSTM**: Dominates contextual generation mapping. Successfully hallucinates longer, coherent combinations while suffering minimally from short-circuit EOS tokens.
- **Attention Integration**: Ironically displayed instability given the small size of the vocabulary space, frequently jumping to end-of-sequence closures too early (e.g., outputting 1-character syllables due to overfitting on local contextual mappings).

2.5 Analysis

- **Vanilla RNN**: At low temperature (0.5), Vanilla RNN tends to be more deterministic, often producing familiar-sounding names but with lower diversity. At high temperature (1.2), the output becomes more creative but may produce unrealistic character combinations (higher failure rate).
- **BLSTM**: At low temperature (0.5), Bidirectional LSTM tends to be more deterministic, often producing familiar-sounding names but with lower diversity. At high temperature (1.2), the output becomes more creative but may produce unrealistic character combinations (higher failure rate).

- **Attention Integration**: At low temperature (0.5), RNN + Attention tends to be more deterministic, often producing familiar-sounding names but with lower diversity. At high temperature (1.2), the output becomes more creative but may produce unrealistic character combinations (higher failure rate).

2.6 Architecture Observations:

- **Vanilla RNN**:The simplest architecture. Tends to capture common name patterns (short suffixes like -a, -i, -an) but struggles with less frequent character sequences due to the vanishing gradient problem. Generated names are often plausible but lack variety.

- **BLSTM**: By processing sequences in both directions during training, the BLSTM builds richer contextual representations. During inference it operates as a standard LSTM decoder. Names tend to be slightly more diverse than Vanilla RNN, with better handling of longer names.

- **Attention Integration**: The attention mechanism allows the model to selectively focus on relevant parts of the input when generating each character. This typically improves coherence — the model avoids drifting into random character sequences mid-name. However, the added complexity also increases the risk of overfitting on a small dataset like ours.